UNIVERSIDAD PRIVADA DEL NORTE
Facultad de Ingeniería | Ingeniería de Sistemas Computacionales

GUÍA HANDS-ON

Semana 2 — Unidad 1

Listas Enlazadas Simples: Ordenamiento, Búsqueda y Mezcla en C#

Curso: Estructura de Datos | Código: SIST1201A | Ciclo: 4.° | Período: 2026-1

Dr. Eric Gustavo Coronel Castillo

1. Información General

Curso	Estructura de Datos
Código	SIST1201A
Semana / Sesión	Semana 2 — Sesión 2
Unidad	Unidad 1: Listas, Pilas y Colas
Período	2026-1
Tiempo estimado	45 minutos (laboratorio)
Docente	Dr. Eric Gustavo Coronel Castillo

2. Logro de la Sesión

Al término de la sesión, el estudiante:

- Implementa el algoritmo de búsqueda secuencial sobre una lista enlazada simple en C#.
- Implementa el algoritmo de ordenamiento burbuja (Bubble Sort) sobre una lista enlazada simple en C#.
- Implementa el algoritmo de mezcla ordenada (Merge) de dos listas enlazadas simples en C#.
- Integra las operaciones anteriores en una aplicación Windows Forms con interfaz gráfica.

3. Prerrequisitos y Herramientas

Antes de iniciar el laboratorio, verifique que cuenta con los siguientes conocimientos y herramientas:

Conocimientos previos	Herramientas necesarias
• Clases y objetos en C# • Constructores y propiedades • Operador de referencia (null) • Estructuras de control (while, do-while) • Concepto de Lista Enlazada Simple (Semana 1)	• Visual Studio 2022 (Community o superior) • .NET 6 o superior • Windows Forms (WinForms) • Git (recomendado para control de versiones)

4. Arquitectura de la Solución

La solución se organiza en cuatro capas conceptuales, siguiendo el principio de Separación de Responsabilidades (SoC) de Robert C. Martin (Clean Code, 2008):

4.1. Diagrama de Clases (Descripción textual)

Clase	Atributos / Propiedades	Métodos
Producto	Nombre: string Precio: double	Producto(nombre, precio) ToString()
Nodo	Dato: Producto Siguiente: Nodo	Nodo(dato)
ListaEnlazada	cabeza: Nodo (privado)	InsertarAlFinal(dato) Recorrer() Buscar(nombre): Nodo OrdenarBurbuja() Mezclar(lista): ListaEnlazada
FormPrincipal (Windows Forms)	lista1: ListaEnlazada lista2: ListaEnlazada dgvProductos: DataGridView	btnAgregar_Click() btnBuscar_Click() btnOrdenar_Click() btnMezclar_Click() ActualizarGrid(lista, dgv)

Nota: La clase Producto encapsula los datos del dominio. La clase Nodo es la unidad estructural de la lista. La clase ListaEnlazada gestiona la colección. El formulario es exclusivamente responsable de la presentación (patrón de separación UI / lógica).

5. Desarrollo Paso a Paso

A continuación, se presenta la secuencia de implementación. Siga cada paso en orden dentro de Visual Studio 2022.

Paso 1 — Crear el Proyecto Windows Forms

Instrucciones

- Abra Visual Studio 2022 y seleccione Crear un nuevo proyecto.
- Elija la plantilla: **Aplicación de Windows Forms (.NET)** y haga clic en Siguiente.
- Nombre del proyecto: **GestionProductos**. Framework: .NET 6.0. Haga clic en Crear.
- En el Explorador de soluciones, agregue tres archivos de clase: Producto.cs, Nodo.cs y ListaEnlazada.cs.

Paso 2 — Clase Producto.cs

La clase Producto representa la entidad de dominio. Encapsula los datos que almacenará cada nodo de la lista enlazada. El método ToString() permite mostrar el objeto de forma legible.

```
// Archivo: Producto.cs
// Responsabilidad: Encapsular los datos del producto (entidad de dominio).

namespace GestionProductos
{
    public class Producto
    {
        // Propiedades con acceso controlado (encapsulamiento)
        public string Nombre { get; set; }
        public double Precio { get; set; }

        // Constructor: inicializa los atributos al crear el objeto
        public Producto(string nombre, double precio)
        {
            Nombre = nombre;
            Precio = precio;
        }

        // Representacion textual del objeto (utilizada en consola y grids)
        public override string ToString()
        {
```

```
            return $"{Nombre} - S/ {Precio:F2}";
        }
    }
}
```

Punto clave: la propiedad Precio es de tipo double, lo que permite comparaciones numéricas directas durante el ordenamiento. Se usa el formato :F2 para mostrar dos decimales.

Paso 3 — Clase Nodo.cs

La clase Nodo es la unidad básica de la lista enlazada. Contiene el dato (un objeto Producto) y la referencia al nodo siguiente. Cuando Siguiente es null, indica el final de la lista.

```
// Archivo: Nodo.cs
// Responsabilidad: Ser la unidad estructural de la lista enlazada.

namespace GestionProductos
{
    public class Nodo
    {
        public Producto Dato { get; set; }      // Datos del nodo
        public Nodo Siguiente { get; set; }     // Referencia al proximo nodo

        public Nodo(Producto dato)
        {
            Dato = dato;
            Siguiente = null;  // Al crearse, no apunta a ningun nodo
        }
    }
}
```

Paso 4 — Clase ListaEnlazada.cs (Operaciones principales)

Esta clase es el núcleo de la guía. Implementa todos los algoritmos de la sesión: recorrido, búsqueda, ordenamiento burbuja y mezcla ordenada. Léala con atención antes de copiarla.

4.1 Estructura base e inserción al final

```
// Archivo: ListaEnlazada.cs
// Responsabilidad: Gestionar la coleccion de nodos y sus operaciones.
namespace GestionProductos
{
    public class ListaEnlazada
    {
        // Atributo privado: solo esta clase conoce el nodo inicial
        private Nodo cabeza;

        public ListaEnlazada()
        {
            cabeza = null;  // Lista vacia al inicio
        }

        // ── INSERTAR AL FINAL ──────────────────────────────────────
        // Complejidad: se recorre toda la lista para llegar al último nodo
        public void InsertarAlFinal(Producto dato)
        {
            Nodo nuevo = new Nodo(dato);
            if (cabeza == null)
            {
                cabeza = nuevo;  // Primer elemento de la lista
                return;
            }
            Nodo actual = cabeza;
            while (actual.Siguiente != null)
            {
                actual = actual.Siguiente;
            }
            actual.Siguiente = nuevo;  // Enlaza el nuevo nodo al final
        }
    }
}
```

4.2 Recorrido de la lista

```
// ── RECORRER ──────────────────────────────────
// Recorre la lista desde la cabeza hasta null
public List<Producto> ObtenerTodos()
{
  List<Producto> resultado = new List<Producto>();
  Nodo actual = cabeza;
  while (actual != null)
  {
    resultado.Add(actual.Dato);
    actual = actual.Siguiente;
  }
  return resultado;
}
```

4.3 Búsqueda secuencial

La búsqueda en una lista enlazada es siempre secuencial (O(n)) porque no existe acceso aleatorio al i-ésimo elemento. Se recorre desde la cabeza comparando el nombre de cada nodo.

```
// ── BUSCAR ────────────────────────────────────────────
// Recorre la lista nodo por nodo comparando el nombre del producto.
// Retorna el nodo encontrado o null si no existe.
// Complejidad: el peor caso peor, cual el elemento esta al final o no existe
public Nodo Buscar(string nombre)
{
  Nodo actual = cabeza;
  while (actual != null)
  {
    // Comparacion sin distincion de mayusculas/minusculas
    if (actual.Dato.Nombre.Equals(nombre,
      StringComparison.OrdinalIgnoreCase))
    {
      return actual;  // Encontrado: retorna el nodo
    }
    actual = actual.Siguiente;
  }
  return null;  // No encontrado
}
```

4.4 Ordenamiento burbuja (Bubble Sort)

El algoritmo de ordenamiento burbuja compara pares de nodos adyacentes e intercambia sus datos (no los punteros) cuando están en el orden incorrecto. Se utiliza la variable hayIntercambio para terminar anticipadamente si la lista ya está ordenada.

```
// ── ORDENAMIENTO BURBUJA ──────────────────────────────────
// Ordena la lista por Precio de forma ascendente.
// Intercambia el contenido (Dato) de los nodos, no los punteros.
public void OrdenarBurbuja()
{
  if (cabeza == null) return;  // Lista vacia: nada que ordenar

  bool hayIntercambio;
  do
  {
    hayIntercambio = false;
    Nodo actual = cabeza;

    while (actual.Siguiente != null)
    {
      // Si el nodo actual tiene mayor precio que el siguiente: intercambiar
      if (actual.Dato.Precio > actual.Siguiente.Dato.Precio)
      {
        // Swap: se intercambia el Dato, no los punteros
        Producto temp = actual.Dato;
        actual.Dato = actual.Siguiente.Dato;
        actual.Siguiente.Dato = temp;
        hayIntercambio = true;
      }
      actual = actual.Siguiente;
    }
  } while (hayIntercambio);  // Repite hasta que no haya intercambios
}
```

4.5 Mezcla ordenada de dos listas (Merge)

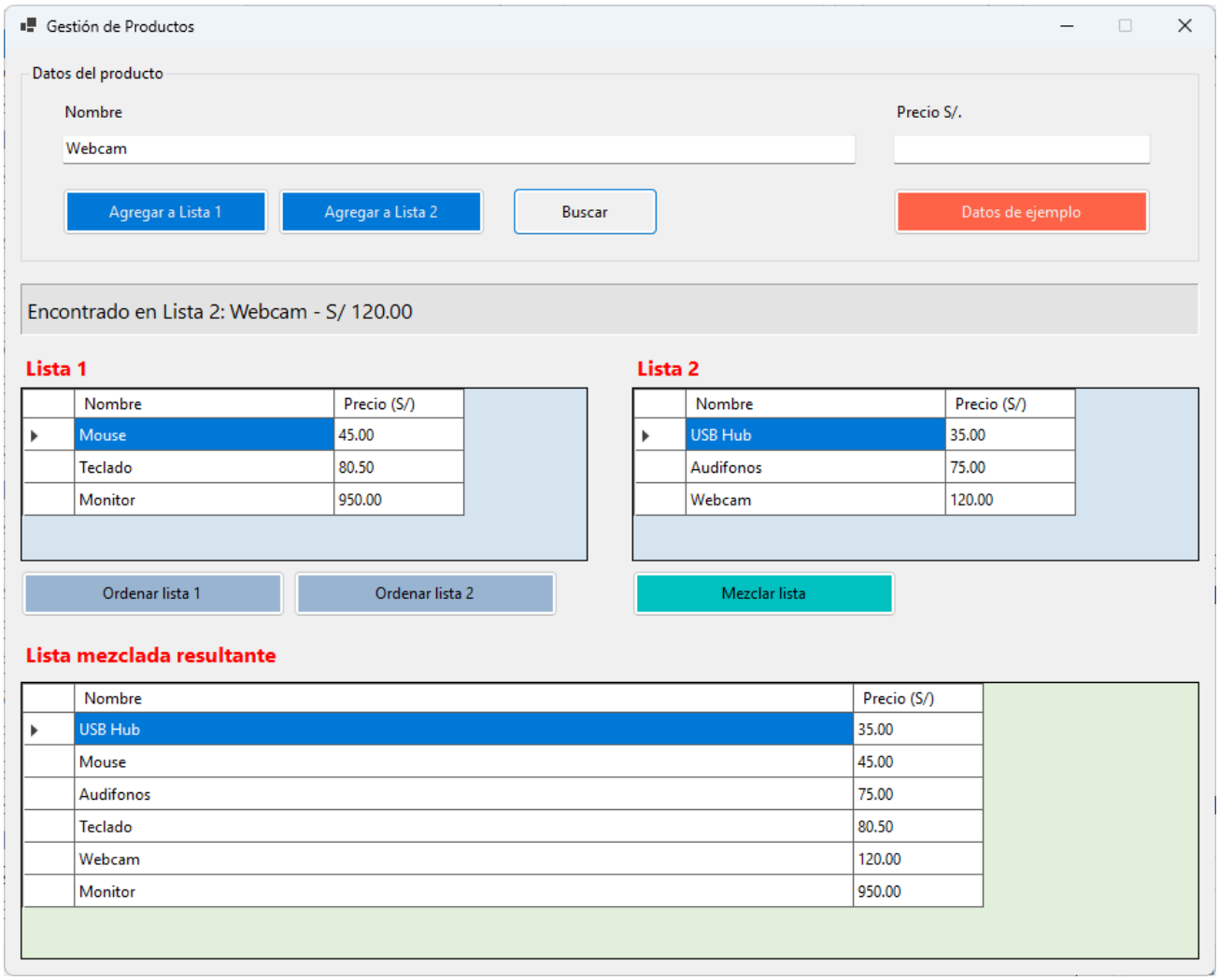
El algoritmo de mezcla recibe una segunda lista como parámetro y combina ambas en una nueva lista resultante, manteniendo el orden ascendente por Precio. Emplea un nodo centinela (dummy) para simplificar la lógica de inserción. Esta es la técnica descrita en Cormen et al. (2022).

```
// ── MEZCLA ORDENADA ──────────────────────────────────────
// Pre-condicion: ambas listas deben estar ordenadas por Precio.
// Retorna una nueva lista con los elementos de ambas en orden ascendente.
public ListaEnlazada Mezclar(ListaEnlazada otraLista)
{
  ListaEnlazada resultado = new ListaEnlazada();
  Nodo pA = this.cabeza;         // Puntero a la lista actual
  Nodo pB = otraLista.cabeza;    // Puntero a la otra lista

  while (pA != null && pB != null)
  {
    if (pA.Dato.Precio <= pB.Dato.Precio)
    {
      resultado.InsertarAlFinal(pA.Dato);
      pA = pA.Siguiente;
    }
    else
    {
      resultado.InsertarAlFinal(pB.Dato);
      pB = pB.Siguiente;
    }
  }
  // Conectar la cola restante de la lista no agotada
  while (pA != null)
  {
    resultado.InsertarAlFinal(pA.Dato);
    pA = pA.Siguiente;
  }
  while (pB != null)
  {
    resultado.InsertarAlFinal(pB.Dato);
    pB = pB.Siguiente;
  }
  return resultado;
}
```

Paso 5 — Diseño del Formulario Windows Forms

El formulario FormPrincipal integra las dos listas y expone las operaciones mediante botones. Siga las instrucciones de diseño visual antes de agregar el código.



5.1 Controles del formulario

Control (Name)	Tipo	Propósito / Text
`txtNombre`	TextBox	Ingresa el nombre del producto
`txtPrecio`	TextBox	Ingresa el precio del producto
`btnAgregar1`	Button	"Agregar a Lista 1"
`btnAgregar2`	Button	"Agregar a Lista 2"
`btnBuscar`	Button	"Buscar"
`btnOrdenar1`	Button	"Ordenar Lista 1"
`btnOrdenar2`	Button	"Ordenar Lista 2"
`btnMezclar`	Button	"Mezclar Listas"
`dgvLista1`	DataGridView	Muestra los productos de Lista 1
`dgvLista2`	DataGridView	Muestra los productos de Lista 2
`dgvMezclada`	DataGridView	Muestra la lista mezclada resultante
`lblResultado`	Label	Muestra mensajes de búsqueda

5.2 Código del formulario FormPrincipal.cs

```
// Archivo: FormPrincipal.cs
// Responsabilidad: Capa de presentacion.
// Conecta la UI con la logica de la lista.

namespace GestionProductos
{
  public partial class FormPrincipal : Form
  {

    // Instancias de las dos listas enlazadas
    private ListaEnlazada lista1 = new ListaEnlazada();
    private ListaEnlazada lista2 = new ListaEnlazada();

    public Form1()
    {
      InitializeComponent();
      ConfigurarGrids();
    }
```

```
// Configura las columnas de los DataGridView
private void ConfigurarGrids()
{
  string[] cols = { "Nombre", "Precio (S/)" };
  foreach (DataGridView dgv in
    new[] { dgvLista1, dgvLista2, dgvMezclada })
  {
    dgv.Columns.Clear();
    dgv.Columns.Add("colNombre", cols[0]);
    dgv.Columns.Add("colPrecio", cols[1]);
    dgv.Columns["colNombre"].Width = (dgv == dgvMezclada ? 600 : 200);
    dgv.Columns["colPrecio"].Width = 100;
    dgv.AllowUserToAddRows = false;
    // Identificar si es el dgvMezclada
    if (dgv == dgvMezclada)
    {
      dgv.BackgroundColor = Color.FromArgb(226, 239, 218); // verde claro
      dgv.ColumnHeadersDefaultCellStyle.BackColor
             = Color.FromArgb(30, 113, 69);
      dgv.ColumnHeadersDefaultCellStyle.ForeColor = Color.White;
    }
    else
    {
      dgv.BackgroundColor = Color.FromArgb(214, 228, 240); // azul claro
      dgv.ColumnHeadersDefaultCellStyle.BackColor
             = Color.FromArgb(46, 117, 182);
      dgv.ColumnHeadersDefaultCellStyle.ForeColor = Color.White;
    }
  }
}
```

```
// Actualiza un DataGridView a partir de una lista enlazada
private void ActualizarGrid(ListaEnlazada lista, DataGridView dgv)
{
  dgv.Rows.Clear();
  foreach (Producto p in lista.ObtenerTodos())
  {
    dgv.Rows.Add(p.Nombre, p.Precio.ToString("F2"));
  }
}

// EVENTOS

private void btnAgregar1_Click(object sender, EventArgs e)
{
  if (!ValidarEntrada(out string nombre, out double precio)) return;
  lista1.InsertarAlFinal(new Producto(nombre, precio));
  ActualizarGrid(lista1, dgvLista1);
  LimpiarEntrada();
}

private void btnAgregar2_Click(object sender, EventArgs e)
{
  if (!ValidarEntrada(out string nombre, out double precio)) return;
  lista2.InsertarAlFinal(new Producto(nombre, precio));
  ActualizarGrid(lista2, dgvLista2);
  LimpiarEntrada();
}
```

```
// Aplicando Early Return o Guard Clause
// Robert C.Martin lo recomienda explícitamente en Clean Code (2008)
// como una forma de reducir la complejidad cognitiva del código.
private void btnBuscar_Click(object sender, EventArgs e)
{
  string nombre = txtNombre.Text.Trim();
  if (string.IsNullOrEmpty(nombre))
  {
    MessageBox.Show("Ingrese un nombre para buscar.",
      "Busqueda", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
  }
  Nodo encontrado = lista1.Buscar(nombre);
  if (encontrado != null)
  {
    lblResultado.Text = $"Encontrado en Lista 1: {encontrado.Dato}";
    return;
  }
  encontrado = lista2.Buscar(nombre);
  if (encontrado != null)
  {
    lblResultado.Text = $"Encontrado en Lista 2: {encontrado.Dato}";
    return;
  }
  lblResultado.Text = "Producto no encontrado en ninguna lista.";
}

private void btnOrdenar1_Click(object sender, EventArgs e)
{
  lista1.OrdenarBurbuja();
  ActualizarGrid(lista1, dgvLista1);
}

private void btnOrdenar2_Click(object sender, EventArgs e)
{
  lista2.OrdenarBurbuja();
  ActualizarGrid(lista2, dgvLista2);
}
```

```
private void btnMezclar_Click(object sender, EventArgs e)
{
  // Precondicion: ambas listas deben estar ordenadas antes de mezclar
  lista1.OrdenarBurbuja();
  lista2.OrdenarBurbuja();
  ActualizarGrid(lista1, dgvLista1);
  ActualizarGrid(lista2, dgvLista2);
  // Mezcla ordenada:
  //     se asume que ambas listas ya estan ordenadas por Precio
  ListaEnlazada mezclada = lista1.Mezclar(lista2);
  ActualizarGrid(mezclada, dgvMezclada);
  lblResultado.Text = "Mezcla completada correctamente.";
}

private void btnDatosEjemplo_Click(object sender, EventArgs e)
{
  // Limpiar listas existentes
  lista1 = new ListaEnlazada();
  lista2 = new ListaEnlazada();

  // Cargar datos en Lista 1
  lista1.InsertarAlFinal(new Producto("Teclado", 80.50));
  lista1.InsertarAlFinal(new Producto("Mouse", 45.00));
  lista1.InsertarAlFinal(new Producto("Monitor", 950.00));

  // Cargar datos en Lista 2
  lista2.InsertarAlFinal(new Producto("USB Hub", 35.00));
  lista2.InsertarAlFinal(new Producto("Webcam", 120.00));
  lista2.InsertarAlFinal(new Producto("Audifonos", 75.00));

  // Reflejar en los DataGridView
  ActualizarGrid(lista1, dgvLista1);
  ActualizarGrid(lista2, dgvLista2);

  // Limpiar la lista mezclada si hubiera un resultado previo
  dgvMezclada.Rows.Clear();

  lblResultado.Text = "Datos de ejemplo cargados correctamente.";
}
```

```
    // AUXILIARES

    private bool ValidarEntrada(out string nombre, out double precio)
    {
      // Inicializa las variables de salida
      nombre = txtNombre.Text.Trim();
      precio = 0;
      // Valida que el nombre no este vacio.
      if (string.IsNullOrEmpty(nombre))
      {
        MessageBox.Show("El nombre no puede estar vacio.",
          "Validacion", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        txtNombre.Focus();
        return false;
      }
      // Valida que el precio sea un numero valido mayor a cero.
      string texto = txtPrecio.Text.Trim().Replace(",", ".");
      bool exito = double.TryParse(texto, out precio);
      if (!exito || precio <= 0)
      {
        MessageBox.Show("Ingrese un precio numerico mayor a cero.",
          "Validacion", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        txtPrecio.Focus();
        return false;
      }
      return true;
    }

    private void LimpiarEntrada()
    {
      txtNombre.Clear();
      txtPrecio.Clear();
      txtNombre.Focus();
    }

  }
}
```

6. Verificación del Funcionamiento

Una vez completada la implementación, ejecute la aplicación (F5) y realice las siguientes pruebas de aceptación:

#	Acción	Resultado esperado
1	Agregar 3 productos a Lista 1 con precios: 80, 30, 55	Los 3 productos aparecen en dgvLista1 en el orden de inserción
2	Agregar 3 productos a Lista 2 con precios: 20, 70, 45	Los 3 productos aparecen en dgvLista2 en el orden de inserción
3	Hacer clic en Ordenar Lista 1	dgvLista1 muestra los productos ordenados: 30, 55, 80
4	Buscar un producto existente en Lista 1	lblResultado muestra: "Encontrado en Lista 1: Nombre - S/ XX.XX"
5	Buscar un producto inexistente	lblResultado muestra: "Producto no encontrado en ninguna lista."
6	Hacer clic en Mezclar Listas	dgvMezclada muestra los 6 productos en orden ascendente de precio
7	Intentar agregar con el campo Nombre vacío	MessageBox de advertencia con mensaje de validación
8	Intentar agregar con precio no numérico	MessageBox de advertencia indicando precio inválido
9	Intentar cargar datos de ejemplo	Las listas deben cargarse con 3 productos cada una.

7. Retos Propuestos

Los siguientes retos tienen como propósito profundizar la comprensión de los algoritmos trabajados. Se presentan en orden ascendente de dificultad. Se recomienda resolverlos de forma individual y sin modificar el código de la guía.

Reto 1 — Nivel Básico (★☆☆)

- Modifique el método OrdenarBurbuja() para que acepte un parámetro booleano ascendente. Si ascendente = true, ordena de menor a mayor precio; si ascendente = false, ordena de mayor a menor.
- Agregue dos botones en el formulario: 'Orden Asc.' y 'Orden Desc.', cada uno con la invocación correspondiente.
- Pista: solo debe cambiar el operador de comparación (> por <) dentro del if de intercambio.

Reto 2 — Nivel Intermedio (★★☆)

- Implemente el método OrdenarInsercion() en la clase ListaEnlazada que ordene la lista por Nombre de producto en orden alfabético ascendente.
- El ordenamiento debe realizarse redirigiendo los punteros Siguiente, no intercambiando datos.
- Agregue un botón 'Ordenar por Nombre' y verifique que la lista se ordena correctamente con al menos 5 productos.
- Pista: el método String.Compare(a, b, StringComparison.OrdinalIgnoreCase) retorna negativo si a < b, cero si a == b, y positivo si a > b.

Reto 3 — Nivel Avanzado (★★★)

- Implemente el método Eliminar(string nombre) que busque y elimine el nodo cuyo Nombre coincida con el parámetro. Considere los tres casos: eliminar el primer nodo (cabeza), un nodo intermedio y el último nodo.
- Agregue un botón 'Eliminar' en el formulario y actualice el DataGridView correspondiente.
- Valide el caso en que el producto no existe e informe al usuario mediante un MessageBox.
- Analice la complejidad temporal del método Eliminar() y documente su respuesta como comentario al inicio del método.
- Pista: necesitará un puntero anterior que vaya un paso detrás del puntero actual para poder redirigir el enlace.

8. Conclusiones de la Sesión

01	La búsqueda en una lista enlazada simple es siempre de complejidad O(n) porque no existe acceso aleatorio. Esta limitación es el costo de la flexibilidad que ofrece la estructura en inserciones y eliminaciones.
02	El ordenamiento burbuja implementado sobre una lista enlazada intercambia los datos (Dato) de los nodos en lugar de sus punteros. Esto simplifica la lógica pero tiene la misma complejidad $O(n^2)$ que su equivalente sobre arreglos.
03	El algoritmo de mezcla ordenada tiene complejidad O(m + n), lo que lo convierte en la operación más eficiente de la sesión. Es la base del algoritmo Merge Sort, uno de los algoritmos de ordenamiento más eficientes con complejidad O(n log n).
04	Implementar manualmente estas operaciones construye el fundamento conceptual para comprender y utilizar con criterio la clase LinkedList<T> de .NET, y para abordar estructuras de mayor complejidad como pilas, colas y grafos.

9. Bibliografía de Referencia

Las siguientes referencias académicas y técnicas sustentan el contenido de esta guía. Se presentan en formato APA (7.ª edición):

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

[2] Weiss, M. A. (2012). Data structures and algorithm analysis in C# (3.ª ed.). Pearson. https://www.pearson.com/en-us/subject-catalog/p/data-structures-and-algorithm-analysis-in-c/P200000003402

[3] Sedgewick, R., y Wayne, K. (2011). Algorithms (4.ª ed.). Addison-Wesley Professional. https://algs4.cs.princeton.edu/home/

[4] Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall. https://www.pearson.com/en-us/subject-catalog/p/clean-code-a-handbook-of-agile-software-craftsmanship/P200000009044

[5] Microsoft. (2024). LinkedList<T> class. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.linkedlist-1

[6] Microsoft. (2024). Guía de programación de C#. Microsoft Learn. https://docs.microsoft.com/es-es/dotnet/csharp/programming-guide/

[7] Hernández Bejarano, M., y Baquero Rey, L. E. (2022). Estructuras de datos: Fundamentación práctica. Biblioteca Virtual UPN. https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230581

Material elaborado para uso académico exclusivo.
Universidad Privada del Norte | Ingeniería de Sistemas Computacionales | Período 2026-1